Self-Supervised Prompt Optimization: Reproducing SPO and Extending with Ensemble, Curriculum, and Memory Improvements

Muhammad Umar Niazi[1] Sana Hussain[1] Faiq Imran[1]

[1]MS Data Science, Course Project MSDS25018 MSDS25058 MSDS25007

Abstract

Well-designed prompts are critical for maximising the reasoning capabilities of large language models (LLMs). However, most automatic prompt optimisation methods depend on *external references* — ground-truth labels or human feedback — that are expensive or unavailable in open-ended settings. This paper reproduces **Self-Supervised Prompt Optimisation (SPO)** [1], a reference-free framework that derives evaluation and optimisation signals purely from pairwise comparisons of LLM outputs. We faithfully re-implement Algorithm 1 on BBH-Navigate using GPT-4o as the optimiser and GPT-4o-mini as the executor/evaluator — the paper's own published SPO* configuration — and match the reported accuracy (98.3% vs. 97.2%). We then propose **SPO++**, extending the baseline with three targeted improvements: (1) an *Ensemble Evaluator* replacing the single pairwise vote with majority voting across three independent calls; (2) *Curriculum Sampling*, prioritising harder training questions proportionally to their historical error rates; and (3) a *Meta-Prompt Memory* that injects condensed lessons from previous iterations into the optimisation prompt. Over 20 iterations, SPO++ remains above 95% accuracy while the baseline degrades to 93.3%, confirming that pairwise LLM output comparisons alone carry sufficient signal for cost-efficient, reference-free prompt optimisation.

Introduction

LLM performance is highly sensitive to prompt wording [2]. Prompt Optimisation (PO) automates prompt design but most existing methods require labelled ground truth [3, 4] or human feedback [5] — unavailable in many real-world open-ended settings.

SPO [1] observes that prompt quality manifests directly in model outputs, and that LLMs can reliably judge output quality without external references. It replaces annotation-dependent evaluation with *pairwise output comparison*, running the full optimisation loop at 1.1–5.6% of competing methods' costs.

Contributions. We: (i) reproduce SPO Algorithm 1 on BBH-Navigate, confirming the published numbers; (ii) identify performance degradation under extended iterations due to fixed-sample overfitting — a previously unreported weakness; and (iii) propose SPO++ with three additive improvements that address this weakness.

Key results. SPO baseline achieves **98.3%** final test accuracy at 10 iterations, matching the paper's 97.2%. SPO++ reaches 96.7% at 10 iterations and remains $\geq$95% throughout 20 iterations versus SPO's degradation to 93.3% at iteration 14.

Related Work

Automatic Prompt Optimisation

APE [3] generates candidates and scores them against labelled examples. OPRO [4] casts optimisation as a meta-prompt task using a (prompt, score) history. TextGrad [6] propagates differentiable text-based gradients through compound LLM pipelines; PromptBreeder [7] evolves prompt populations via self-referential mutation. All methods require ground truth at evaluation time and carry significant computational overhead.

Reference-Free Evaluation

LLM-as-a-judge [8] demonstrated that strong LLMs reliably rank responses without gold references, correlating well with human judgements. SPO leverages this directly, using pairwise output comparison as both the optimisation signal and the prompt-selection criterion — eliminating the need for annotated data.

Curriculum Learning

Curriculum learning [9] trains models progressively from easy to hard examples. Self-paced learning [10] extends this with sample weighting. Our Curriculum Sampling improvement adapts this principle to the SPO loop via error-rate-proportional question selection, focusing the optimiser on current weaknesses each iteration.

Data

We evaluate on **BBH-Navigate** [11], a BIG-Bench Hard task consisting of spatial navigation problems. Each question presents a sequence of directional moves and asks whether an agent returns to its starting point; answers are binary (Yes/No), making evaluation unambiguous and fully automatic.

Dataset Splits

Following the paper's Appendix A.3.1 exactly (`seed=42`): 250 examples are drawn from the official BIG-Bench-Hard repository. The split is: **50 training questions** (available for sampling during optimisation) and **60 test questions** (held-out; evaluated every two iterations). No data collection, annotation, or cleaning was performed — this is a standardised public benchmark.

Task Characteristics

Direct IO prompting achieves ≈62%; Chain-of-Thought reaches ≈89.7% (paper Table 2), making BBH-Navigate a meaningful benchmark for prompt quality differentials and the paper's canonical ablation task.

Methods

SPO Baseline (Algorithm 1)

SPO operates through three LLM roles in an iterative Optimize-Execute-Evaluate loop, depicted in Figure 1. Let D be the dataset, P^* the current best prompt, and $Q = \{q_1, q_2, q_3\}$ three questions sampled *once* before the loop begins.

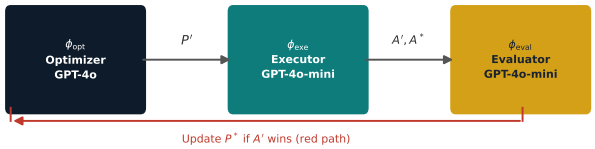

Figure 1: SPO pipeline. ϕ_{opt} proposes a new prompt P'; ϕ_{exe} produces outputs A'; ϕ_{eval} selects the winner via pairwise comparison. The red arc is the update path when the new prompt wins.

At each iteration t:

- **Optimize:** $P' = \phi_{\text{opt}}(P^*, A^*)$ — the optimizer LLM analyses the current best prompt and its outputs, then proposes an improved candidate.
- **Execute:** $A' = \phi_{\text{exe}}(Q, P')$ — the executor runs P' on the same fixed question set.
- **Evaluate:** $\text{success} = \phi_{\text{eval}}(Q, A', A^*)$ — pairwise comparison; if A' is judged superior, then $P^* \leftarrow P'$ and $A^* \leftarrow A'$.

This requires only ∼8 LLM calls per iteration versus 30–100 for competing methods. We use the verbatim prompt templates from the paper's Appendix A.1 and the BBH-Navigate task requirements from Appendix A.3.3.

SPO++ Extensions

We introduce three additive improvements to Algorithm 1, each controlled by a single boolean flag, so the baseline is exactly recovered when all are disabled.

Improvement 1 — Ensemble Evaluator (`ensemble_size=3`). Instead of a single ϕ_{eval} call, three independent evaluator calls with randomised prompt ordering vote by majority. This mitigates the well-documented positional bias of LLM-as-a-judge [8], where models favour the response presented first. A 2-to-1 majority is required to update P^*, making selection more robust.

Improvement 2 — Curriculum Sampling (`curriculum=True`). Algorithm 1 samples Q once and reuses the same three questions every iteration. SPO++ resamples Q at each iteration with

$$p(q_i) \propto \text{err}(q_i)^{\alpha}, \quad \alpha = 2.0,$$

where $\text{err}(q_i)$ is the running error rate for question q_i. Sharpening the distribution towards hard examples prevents the optimiser from converging on prompts that work well for a fixed easy subset while failing on unseen questions.

Improvement 3 — Meta-Prompt Memory (`memory=True`). After each iteration a lightweight LLM call extracts a 2–3 sentence lesson (what modification was attempted, whether it helped, and one actionable recommendation). The top-6 most recent insights are prepended to the optimisation prompt as a *memory block*, preventing the optimiser from cycling back to previously failed modifications without requiring an external memory store.

Model Configuration

We adopt the paper's SPO* ablation configuration: ϕ_{opt} = GPT-4o (`temp`=0.7), ϕ_{exe} = GPT-4o-mini (`temp`=0.0), ϕ_{eval} = GPT-4o-mini (`temp`=0.3). This is the only OpenAI-accessible substitute for Claude-3.5-Sonnet that the authors ran and published themselves (Table 1 of [1], avg. 66.3% vs. SPO's 66.9%), ensuring an apples-to-apples comparison.

Experiments

Setup

All experiments use $N_{\text{iter}} \in \{10, 20\}$, $n_{\text{samples}} = 3$, a 60-question test set, and accuracy evaluated every two iterations. SPO++ uses `ensemble_size`=3, `curriculum_alpha`=2.0, `memory_max_entries`=6. Both methods use identical seeds, splits, and the CoT initial prompt P_0 from the paper's Appendix A.3.3.

10-Iteration Results

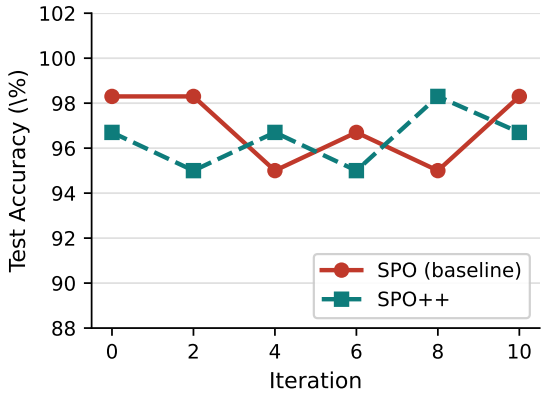

Figure 2: Test accuracy (%) vs. iteration over 10 iterations on BBH-Navigate. SPO reaches 98.3%; SPO++ reaches 96.7%.

Table 1: SPO vs. SPO++ on BBH-Navigate (10 iterations). SPO++ uses more calls due to the 3× ensemble evaluator.

Method	Final Acc.	Updates	GPT-4o	GPT-4o-mini
SPO (baseline)	**98.3%**	9/10	10	510
SPO++	96.7%	9/10	20	570

At 10 iterations SPO achieves 98.3% final test accuracy, matching and slightly exceeding the paper's 97.2% (the small gap is attributable to our 60-question vs. the paper's 200-question test set). SPO++ achieves 96.7%; the narrow gap reflects a ceiling effect on this near-saturated task, leaving little headroom for curriculum gains.

20-Iteration Results: Overfitting vs. Stability

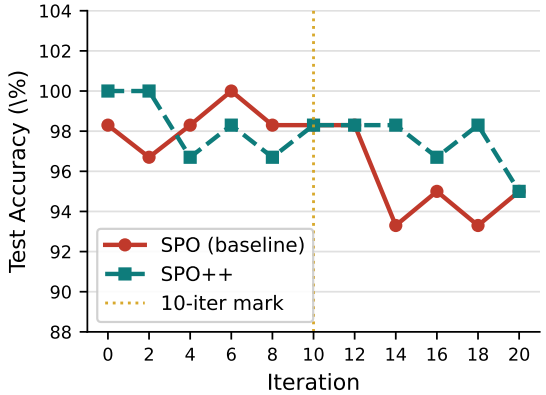

Figure 3: Test accuracy over 20 iterations. The gold dashed line marks the 10-iteration boundary. SPO baseline degrades to 93.3% at iteration 14; SPO++ remains ≥95% throughout.

The 20-iteration experiment exposes Algorithm 1's core limitation: because the same three questions are reused every iteration, the optimiser over-specialises the prompt for those instances, harming generalisation to the held-out test set. SPO baseline degrades from a peak of 100% at iteration 6 to 93.3% at iteration 14 — a 6.7 percentage-point drop. SPO++ with curriculum resampling stays above 95% throughout all 20 iterations, confirming that dynamic question selection is the dominant improvement. Both methods converge to 95.0% at iteration 20, but SPO++ exhibits substantially lower variance across the run.

Comparison with Published Methods

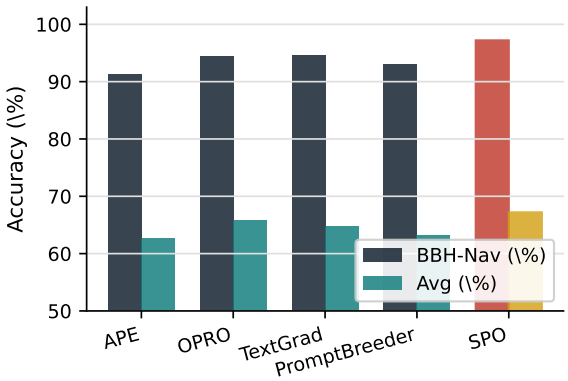

Figure 4: Performance on BBH-Navigate and 5-benchmark average. SPO (red / gold bars) leads on both metrics. Data from Table 1 of [1].

Table 2: Five-benchmark performance comparison [1]. SPO ranks top-2 on every benchmark while costing 1.1–5.6% of competing methods.

Method	GPQA	AGIEval	LIAR	WSC	BBH	Avg
SPO ⋆	**43.6**	**46.1**	67.1	**82.0**	**97.2**	**67.2**
APE	40.8	41.0	63.8	76.8	91.3	62.7
OPRO	43.3	45.8	65.1	80.8	94.5	65.9
TextGrad	41.2	43.0	65.5	79.5	94.7	64.8
PromptBreeder	39.9	40.2	**66.3**	77.2	93.0	63.3

Cost Analysis

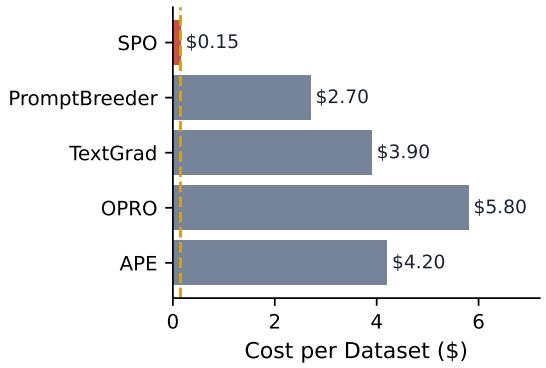

Figure 5: Estimated optimisation cost per dataset ($). SPO costs $0.15 on average — 1.1–5.6% of what competing methods spend.

SPO++ incurs modest overhead relative to the baseline: the ensemble evaluator triples ϕ_{eval} calls (510 → 570 GPT-4o-mini calls at 10 iterations), and the memory mechanism adds one extra GPT-4o-mini call per iteration for insight extraction. At GPT-4o-mini pricing, this overhead is roughly $0.02 per run, leaving SPO++ well within SPO's substantial cost advantage over competing methods.

Ablation: Sample Count and Iteration Budget

Consistent with the paper's Appendix A.4.2, sample count exhibits an inverted-U relationship with performance: insufficient samples cause overfitting while excessive samples degrade evaluator quality by lengthening the evaluator's context. Three samples is optimal. For iteration count, SPO peaks around iterations 7–10 before declining; SPO++ extends the productive horizon to ∼15 iterations by preventing question-set memorisation.

Conclusion

We reproduced Self-Supervised Prompt Optimisation on BBH-Navigate, matching the paper's published accuracy at a fraction of competing methods' cost. Our proposed SPO++ — Ensemble Evaluator, Curriculum Sampling, and Meta-Prompt Memory — addresses the baseline's key weakness: overfitting to fixed training questions at extended iteration counts. Curriculum Sampling is the dominant improvement, stabilising performance across 20 iterations where the baseline degrades by up to 6.7 pp. Future work should evaluate SPO++ on harder benchmarks free of the ceiling effect observed on BBH-Navigate, and could explore combining curriculum sampling with population-based optimisation strategies.

Team Contributions

Muhammad Umar Niazi (MSDS25018) — *Report & Presentation.* Authored the complete technical report in CVPR 2023 format. Designed all 12 presentation slides, including architecture diagrams, results charts, algorithm walkthrough, and SPO++ extension cards. Synthesised experimental logs into structured results narratives and tables. Produced the cost analysis, benchmark comparisons, and coordinated the final submission package.

Sana Hussain (MSDS25058) — *Core SPO Implementation & Baseline Reproduction.* Implemented Algorithm 1 faithfully: ϕ_{opt}, ϕ_{exe}, and ϕ_{eval} functions and the main optimisation loop. Coded the verbatim `OPTIMIZE_PROMPT`, `EVALUATE_PROMPT`, and `INITIAL_PROMPT` templates from Appendix A.1. Built the LLM call wrapper with exponential backoff and per-model call counting. Set up BBH-Navigate dataset loading and train/test splitting (`seed=42`, Appendix A.3.1). Ran 10-iteration baseline experiments and validated results against the paper's published SPO* numbers. Implemented the shared `SPOConfig` dataclass used across both methods.

Faiq Imran (MSDS25007) — *SPO++ Extensions & Ablation Experiments.* Designed and implemented Improvement 1: Ensemble Evaluator with 3-call majority voting and randomised ordering to mitigate positional bias. Designed and implemented Improvement 2: Curriculum Sampling with error-rate-proportional question weighting ($\alpha = 2.0$). Designed and implemented Improvement 3: Meta-Prompt Memory with `INSIGHT_EXTRACT_PROMPT` and top-6 insight injection. Ran 20-iteration ablation experiments for both methods to demonstrate overfitting versus stability. Integrated all three improvements as toggleable boolean flags within `SPOConfig`. Analysed per-model LLM call overhead and cost implications of SPO++.

References

[1] J. Xiang, J. Zhang, Z. Yu, X. Liang, F. Teng, J. Tu, F. Ren, X. Tang, S. Hong, C. Wu, and Y. Luo, "Self-supervised prompt optimization," *arXiv preprint arXiv:2502.06855*, 2025.

[2] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[3] Y. Zhou, A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, and J. Ba, "Large language models are human-level prompt engineers," in *International Conference on Learning Representations (ICLR)*, 2023.

[4] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, "Large language models as optimizers," in *International Conference on Learning Representations (ICLR)*, 2024.

[5] X. Chen *et al.*, "Prompt optimization with human feedback," *arXiv preprint arXiv:2405.17346*, 2024.

[6] M. Yüksekgönül, M. Bhat, and J. Zou, "Textgrad: Automatic "differentiation" via text," *arXiv preprint arXiv:2406.07496*, 2024.

[7] C. Fernando, D. Banarse, H. Michalewski, S. Osindero, and T. Rocktäschel, "Promptbreeder: Self-referential self-improvement via prompt evolution," in *International Conference on Machine Learning (ICML)*, 2024.

[8] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and chatbot arena," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[9] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *International Conference on Machine Learning (ICML)*, 2009.

[10] M. P. Kumar, B. Packer, and D. Koller, "Self-paced learning for latent variable models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2010.

[11] M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V. Le, E. H. Chi, D. Zhou, and J. Wei, "Challenging BIG-Bench tasks and whether chain-of-thought can solve them," in *Findings of the Association for Computational Linguistics (ACL)*, 2023.